Variables
Data types
Operations

CSE 101 Section 03

Introduction to Algorithms and Computing

Fall 2022

© Royal Military College of Canada

What is a Variable?

- **Variables** are used to store information to be referenced and manipulated in a computer program.

- **Variables** also provide a way of labeling data with a descriptive name, so our programs can be understood more clearly by the reader and ourselves.

- *Think of* **variables** as *containers* that hold information. Their sole purpose is to label and store data in memory. This data can then be used throughout your program.

Example

```
% Total price calculator


price_itemA = 20
quantity_itemA = 2


price_itemB = 10;
quantity_itemB = 3;


total = (price_itemA * quantity_itemA) + (price_itemB * quantity_itemB)


total
```

Example

```
% Total price calculator

price_itemA = 20
quantity_itemA = 2

price_itemB = 10;
quantity_itemB = 3;

total = (price_itemA * quantity_itemA) + (price_itemB * quantity_itemB)

total
```

variable name

variable value

used to suppress output

variable is referenced in RHS of calculation

Variable Naming Rules

- A valid variable name starts with a letter, followed by letters, digits, or underscores.

- MATLAB is case sensitive, so `A` and `a` are *not* the same variable.

- You cannot define variables with the same names as MATLAB keywords, such as `if` or `end`. For a complete list, run the `iskeyword` command.

Examples

```
radius_ = 1.3

_diameter = 2.6

a_b = 2

aName123 = "name"

else = 0

Radius = 5
```

Examples

```
radius_ = 1.3
```

✓

```
_diameter = 2.6
```

✗

```
a_b = 2
```

✓

```
aName123 = "name"
```

✓

```
else = 0
```

✗

```
Radius = 5
```

✓

Data Types

A data type, in programming, is a classification that specifies

- which type of values a variable can hold and
- what type of mathematical, relational or logical operations can be applied to it without causing an error.

Example: Integers

- Type of values: whole numbers (e.g. 3, 13, 345)
- Types of operations: addition, multiplication, division, etc.

Data Types in Matlab

- In terms of *syntax*, you do not need to declare the type of a variable. It is implicitly induced from the value assigned to the variable

```
a_b = 2
```

← interpreted as integer

```
aName123 = "name"
```

← interpreted as string

Data Types in Matlab

- There are different types of data that can be stored in variables, some common data types are:
  - *Numbers* (floating point, integer)
  - *Characters* - a single (ASCII??) character
  - *Strings* - a grouping of characters
  - *Boolean* - a value of True (1) or False (0),

Examples

```
anInteger = 5

aFloat = 0.3

aCharacter = 'a'

aString = ‘Matlab is fun!’

aTrueBoolean = true

aFalseBoolean = false

anArray = [1 2 3]

a2DArray = [5 9; 2 1]

% and much more
```

Examples

```
anInteger = 5

aFloat = 0.3

aCharacter = 'a'

aString = ‘Matlab is fun!’

aTrueBoolean = true

aFalseBoolean = false

anArray = [1 2 3]

a2DArray = [5 9; 2 1]

% and much more
```

Matlab special keywords

$[1 \quad 2 \quad 3]$

$\begin{bmatrix} 5 & 9 \\ 2 & 1 \end{bmatrix}$

Operations

The MATLAB language uses many common operators and special characters that you can use to perform simple operations on variables of any type.

- Arithmetic Operations: Addition, subtraction, multiplication, division, power, rounding
- Relational Operations: Value comparisons
- Logical (Boolean) Operations: True or false conditions
- Set Operations: Unions, intersection, set membership
- And more …

Operations: Arithmetic

- Addition
  - Example:

+	Add numbers, append strings

- Subtraction
  - Example:

-	Subtraction

- Multiplication
  - Example:

.*	Multiplication
*	Matrix multiplication

- Division
  - Example:

./	Right array division
.\	Left array division

Operations: Arithmetic (examples)

```
a = [1 1];
b = [2 2];
a + b
```

```
"Matlab " + "is fun"
```

```
3 * 3 % equivalent to 3 .* 3
```

```
[1 3] .* [2 3] % try [1 3] * [2 3]. Got an error? (i.e. red text). Try to fix it!
```

Operations: Relational

- Relational operators compare the elements in two arrays and return logical `true` or `false` values to indicate where the relation holds.

==	Determine equality	3 == 2
>=	Determine greater than or equal to	2 >= 1.5
>	Determine greater than	2 > 2
<=	Determine less than or equal to	5 <= 7
<	Determine less than	3 < 3
~=	Determine inequality	3 ~= 3

Operations: Relational (examples)

```
3 == 2

2 >= 1.5

2 > 2

5 <= 7

3 < 3

3 ~= 3
```

Operations: Relational (examples)

```
3 == 2          False

2 >= 1.5        True

2 > 2           False

5 <= 7          True

3 < 3           False

3 ~= 3          False
```

Aside: Truth tables

AND &

a	b	a & b
0	0	0
0	1	0
1	0	0
1	1	1

OR |

a	b	a | b
0	0	0
0	1	1
1	0	1
1	1	1

Exclusive OR

a	b	xor(a, b)
0	0	0
0	1	1
1	0	1
1	1	0

NOT ~

a	~a
0	1
0	1
1	0
1	0

Notes:

- To execute a logic AND in Matlab → &
- To execute a logic OR in Matlab → |
- To execute a logic NOT in Matlab → ~
- To execute a logic XOR in Matlab → xor(A,B)

- All values except for 0 are considered true; A value of 0 is considered false
- In Matlab we can use the reserved words `true` and `false` for these values
- Matlab (as well as other languages have "short-circuit" operators)
  - && for AND
  - || for OR

Operations: Logical (Boolean)

- Matlab represents Boolean data using the logical data type.
- This data type represents true and false states using the numbers `1` and `0`, respectively.

Short-circuit &&, ||	Logical operations with short-circuiting
&	Find logical AND
~	Find logical NOT
|	Find logical OR
xor	Find logical exclusive-OR
all	Determine if all array elements are nonzero or true
any	Determine if any array elements are nonzero
false	Logical 0 (false)
find	Find indices and values of nonzero elements
islogical	Determine if input is logical array
logical	Convert numeric values to logicals
true	Logical 1 (true)

Operations: Logical (Boolean) Examples

```
% example of shirt circuit &&
b = 1;
a = 20;
x = (b ~= 0) && (a/b > 18.5)
b = 0;
x = (b ~= 0) && (a/b > 18.5)
% example of & and |
A = [5 7 0; 0 2 9; 5 0 0]
B = [6 6 0; 1 3 5; -1 0 0]
A & B
A | B
% example of xor
A = 1
B = 2
C = xor(A,B)
B = 0
C = xor(A,B)
```

Practice

- The documentation of the Matlab operators is available here: https://www.mathworks.com/help/matlab/operators-and-elementary-operations.html

- Once you choose a category, you can click on an operator to see more details and examples.

- Practice by trying some of the provided examples in your Matlab editor, and attempt to predict what the output will be.

Precedence of Operators

The precedence rules for MATLAB operators are shown in this list, ordered from highest precedence level to lowest precedence level:

1. Parentheses ()
2. Transpose (.'), power (.^), complex conjugate transpose ('), matrix power (^)
3. Power with unary minus (.^-), unary plus (.^+), or logical negation (.^~) as well as matrix power with unary minus (^-), unary plus (^+), or logical negation (^~).
4. Unary plus (+), unary minus (-), logical negation (~)
5. Multiplication (.*), right division (./), left division (.\), matrix multiplication (*), matrix right division (/), matrix left division (\)
6. Addition (+), subtraction (-)
7. Colon operator (:)
8. Less than (<), less than or equal to (<=), greater than (>), greater than or equal to (>=), equal to (==), not equal to (~=)
9. Element-wise AND (&)
10. Element-wise OR (|)
11. Short-circuit AND (&&)
12. Short-circuit OR (||)

Example

Find the value of x:

- a=2;
- b=1;
- x= (1+a*b)^(2+1) + 4/2+a

True or False?:

- a=2;
- b=1;
- a<b & b>=a

Next

Selection